

**PROJECT REPORT OF INDUSTRY ORIENTED HANDS ON EXPERIENCE
(IOHE)**

ON

Secure Microservices E-Commerce Platform

submitted in partial fulfilment of the requirements for the award of degree of

BACHELOR OF ENGINEERING

In

COMPUTER SCIENCE AND ENGINEERING

Submitted by:

Guntas Singh (2210990334)

Aditi Goel (2210990053)

Madhav Garg (2210991863)

Hitaishi (2210991666)

Supervised By:

Dr. Gurpreet Singh

Associate Professor and Placement Incharge,

CSE Department, Chitkara University

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

**CHITKARA UNIVERSITY INSTITUTE OF ENGINEERING AND
TECHNOLOGY**

CHITKARA UNIVERSITY, PUNJAB, INDIA



CONTENTS

S.No.	Title	Page No.
	Acknowledgement	iv
	Abstract	v
1	Introduction	1
2	Methodology	2
3	Tools and Technologies	5
4	Implementation	7
5	Results / Outcomes	9
6	Performance Analysis	10
7	Conclusion and Future Scope	11
8	Advantages of the System	12
9	Limitations of the System	14
10	Real-World Applications	15
	References	16
	Appendices	17

DECLARATION

We hereby certify that the work which is being presented in the project report entitled "Secure Microservices E-Commerce Platform" in partial fulfilment of requirement for the award of the degree of Bachelor of Engineering (Computer Science and Engineering) submitted in the department of Computer Science and Engineering at Chitkara University Institute of Engineering and Technology, Chitkara University, Punjab, India, is an authentic record of my own work carried out under the supervision of Dr. Gurpreet Singh. The matter presented in this project report has not been submitted in any other university/institute for the award of any degree.

Place: Rajpura

Date: 28/04/2026

This is to certify that the above statement made by the candidate is correct to the best of my knowledge and belief.

(Dr. Gurpreet Singh)

Associate Professor and Placement Incharge,

Department of Computer Science and Engineering

Chitkara University Institute of Engineering and Technology,

Chitkara University, Punjab, India

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to our mentor Dr. Gurpreet Singh for his continuous guidance, valuable suggestions, and support throughout the development of this project. His insights helped us understand complex technical concepts and successfully implement the system.

We also thank Chitkara University for providing the necessary infrastructure and learning environment. Finally, we appreciate the teamwork and dedication of all group members that contributed to the successful completion of this project.

ABSTRACT

The rapid advancement of digital technologies and the exponential growth of online commerce have significantly increased the demand for highly scalable, secure, and reliable backend systems. Traditional monolithic architectures, although widely used in earlier systems, face several limitations such as tight coupling, difficulty in scaling, complex maintenance, and increased risk of system failure. These challenges make them unsuitable for modern, high-demand applications such as e-commerce platforms.

This project presents the design and implementation of a microservices-based e-commerce backend system that addresses these limitations by adopting a distributed and modular architecture. The system is composed of multiple independent services, including User Service, Product Service, Order Service, and Payment Service. Each service is responsible for a specific functionality and communicates with other services through REST APIs, ensuring loose coupling and independent scalability.

An API Gateway is used as a central entry point for routing client requests to appropriate services, while service discovery is managed using Eureka Server. Security is implemented using JSON Web Tokens (JWT) and role-based access control to ensure safe authentication and authorization mechanisms. The system also integrates an AI-powered chatbot based on Retrieval-Augmented Generation (RAG) and Large Language Models (LLMs) to provide intelligent and real-time customer assistance.

The project demonstrates key features such as fault tolerance, scalability, and maintainability, making it suitable for real-world deployment. Extensive testing, including unit testing and API validation, has been performed to ensure system reliability. Furthermore, the originality and innovation of the developed software have been recognized through copyright filing under the category of computer software.

1. Introduction

1.1. Background

E-commerce platforms have become an essential component of the global digital ecosystem. With the increasing number of users and transactions, systems must be designed to handle high loads efficiently. Traditional monolithic architectures often fail to meet these demands due to their rigid structure and limited scalability.

1.2. Need for Microservices

Microservices architecture divides applications into smaller independent services that can be developed, deployed, and scaled individually. This improves flexibility, reduces downtime, and enhances performance.

1.3. Problem Statement

Traditional systems suffer from the following issues:

- Tight coupling
- Poor scalability
- Difficult maintenance
- Security issues
- Lack of intelligent support systems

1.4. Objectives

- Develop a scalable microservices-based backend
- Implement secure authentication
- Enable efficient service communication
- Integrate AI chatbot
- Ensure system reliability through testing

2. Methodology

The development of the Secure Microservices-Based E-Commerce Platform follows a structured and systematic methodology to ensure scalability, reliability, and efficient system design. The methodology covers all phases of software development, from requirement analysis to testing and validation.

2.1. Requirement Analysis

The first phase of the project involved identifying and analyzing the system requirements. The objective was to determine the essential functionalities required in a modern e-commerce platform.

Functional Requirements

- User registration and login
- Product listing and inventory management
- Order placement and tracking
- Secure payment processing
- AI-based chatbot for customer support

Non-Functional Requirements

- Scalability to handle high user traffic
- Security for protecting user data
- Reliability and fault tolerance
- Maintainability for future enhancements

This phase helped define the project scope and provided a clear roadmap for development.

2.2. System Design

After requirement analysis, the system architecture was designed using a microservices-based approach. The application was divided into independent services, each responsible for a specific functionality. The system design includes:

- API Gateway for routing client requests
- Eureka Server for service discovery

- Microservices such as User, Product, Order, and Payment services
- Separate databases for each service to ensure data isolation

This modular design ensures loose coupling and allows independent scaling of services.

2.3. Microservices Development

Each microservice was developed independently using Spring Boot. The development focused on ensuring that each service performs its designated function efficiently.

- User Service: Handles user authentication and authorization using JWT
- Product Service: Manages product details and inventory
- Order Service: Processes orders and maintains order records
- Payment Service: Handles transaction processing and validation

Each service exposes REST APIs to enable communication between services.

2.4. API Gateway Integration

The API Gateway acts as the central entry point for all client requests. Instead of interacting directly with multiple services, clients communicate through the gateway. The API Gateway performs:

- Request routing to appropriate services
- Authentication validation using JWT tokens
- Logging and monitoring of API requests

This simplifies client interaction and enhances system security.

2.5. Service Discovery Mechanism

Eureka Server is used for dynamic service discovery. Each microservice registers itself with the Eureka Server upon startup. This enables:

- Dynamic location of services without hardcoded URLs
- Easy addition and scaling of services
- Simplified configuration management

2.6. Security Implementation

Security is implemented using JSON Web Tokens (JWT) and role-based access control.

- Users are authenticated through login credentials
- A JWT token is generated upon successful authentication
- The token is used to access protected endpoints
- Role-based access restricts sensitive operations

This ensures secure communication and prevents unauthorized access.

2.7. Testing and Validation

Testing was performed at multiple levels to ensure system correctness and reliability.

- Unit Testing: Conducted using JUnit and Mockito to test individual components
- Integration Testing: Ensured proper communication between microservices
- API Testing: Performed using Postman to validate endpoints
- End-to-End Testing: Verified complete workflows from login to payment

These testing strategies ensured that the system performs reliably under different conditions.

3. Tools and Technologies

The development of the Secure Microservices E-Commerce Platform involves the use of a variety of modern tools and technologies that enable scalability, security, and efficient system design. Each technology was carefully selected based on its capability to support distributed system architecture and real-world application requirements.

Spring Boot:

Spring Boot is a powerful Java-based framework used for developing microservices. It simplifies application development by providing built-in configurations and reducing boilerplate code. In this project, Spring Boot is used to create independent services such as User Service, Product Service, Order Service, and Payment Service. Its embedded server support allows each service to run independently, which is essential for microservices architecture.

Spring Security:

Spring Security is used to implement authentication and authorization mechanisms. It provides robust security features such as password encryption, session management, and access control. In this system, Spring Security is integrated with JWT to ensure that only authenticated users can access protected resources.

JWT (JSON Web Token):

JWT is used for secure communication between client and server. When a user logs in successfully, a token is generated and sent to the client. This token is then used for all subsequent requests, ensuring stateless authentication. JWT enhances security by preventing unauthorized access and eliminating the need for session storage.

Eureka Server:

Eureka Server is used for service discovery in the microservices architecture. Each microservice registers itself with the Eureka Server, allowing other services to locate it dynamically. This eliminates the need for hardcoded service URLs and improves system flexibility.

API Gateway:

The API Gateway acts as a central entry point for all client requests. It routes incoming requests to the appropriate microservices and handles cross-cutting concerns such as authentication, logging, and request filtering. This improves system organization and simplifies client interaction.

MySQL:

MySQL is used as the relational database for storing application data. Each microservice maintains its own database, ensuring data isolation and reducing dependency between services. This approach improves scalability and aligns with microservices principles.

JUnit and Mockito:

JUnit is used for unit testing, while Mockito is used for mocking dependencies. These tools help in testing individual components of the system to ensure correctness and reliability. Unit testing ensures that each service functions independently as expected.

Postman:

Postman is used for API testing. It allows developers to send requests to API endpoints and verify responses. In this project, Postman is used to test authentication, product retrieval, order processing, and payment APIs under various conditions.

Large Language Models (LLM):

LLMs are used in the chatbot module to generate intelligent responses. These models process user queries and generate context-aware replies, improving user experience.

Retrieval-Augmented Generation (RAG):

RAG enhances the chatbot by combining information retrieval with language generation. It retrieves relevant data and generates accurate responses, making the chatbot more effective and reliable.

Overall, the combination of these tools and technologies enables the development of a secure, scalable, and efficient microservices-based e-commerce platform.

4. Implementation

The system is implemented using a distributed microservices architecture where each service is independently developed and deployed.

User Service:

The User Service is responsible for handling user registration, login, and authentication. It uses Spring Security and JWT to ensure secure access. Upon successful login, a JWT token is generated and used for all subsequent requests.

Product Service:

This service manages product-related operations such as adding new products, updating existing ones, and retrieving product details. It also maintains inventory and ensures availability checks before order placement.

Order Service:

The Order Service handles the complete lifecycle of an order. It communicates with the Product Service to validate product availability and with the Payment Service to confirm transaction status before finalizing the order.

Payment Service:

This service processes payments and handles transaction validation. It ensures secure handling of financial data and supports refund functionality in case of failed transactions.

API Gateway:

The API Gateway acts as a central routing layer. It forwards client requests to the appropriate microservices and ensures authentication validation using JWT tokens.

Eureka Server:

Eureka Server enables dynamic service discovery. Each service registers itself with Eureka, allowing other services to discover and communicate without hardcoded URLs.

AI Chatbot Module:

The chatbot module integrates Retrieval-Augmented Generation (RAG) with Large Language Models (LLMs). It retrieves relevant context and generates intelligent responses to user queries, improving customer interaction.

Database Design:

Each service maintains its own database to ensure loose coupling. This design reduces interdependency and enhances scalability.

5. Results / Outcomes

The developed system successfully demonstrates the effectiveness of a microservices-based architecture in building a scalable and secure e-commerce backend platform. Each service in the system operates independently, which allows efficient handling of user requests and enables horizontal scaling when required. This modular approach ensures that the failure of one service does not affect the overall system, thereby improving system reliability and fault tolerance.

The implementation of JWT-based authentication ensures that only authorized users can access protected resources. Role-based access control further enhances security by restricting access to sensitive operations such as order management and payment processing. This makes the system robust against unauthorized access and potential security threats.

The order and payment workflow has been designed to function seamlessly across multiple services. The Order Service effectively communicates with the Product Service to verify product availability and with the Payment Service to confirm successful transactions. This coordinated interaction ensures consistency and reliability in the overall transaction process.

The integration of the AI chatbot module adds significant value to the system by improving user interaction. The chatbot is capable of providing real-time responses to user queries, assisting in product selection, and enhancing the overall user experience. This demonstrates the practical application of AI in modern e-commerce systems.

Testing results indicate that the system performs efficiently under different scenarios. API responses were validated using Postman, and unit testing ensured correctness of individual components. The system was able to handle various edge cases such as invalid inputs, authentication failures, and transaction errors effectively.

Overall, the results confirm that the system meets its objectives of scalability, security, and reliability, making it suitable for real-world e-commerce applications.

6. Performance Analysis

The system was evaluated based on scalability, response time, and reliability.

Scalability:

Due to the microservices architecture, each service can be scaled independently. This ensures efficient handling of increased user load.

Response Time:

API responses were tested using Postman. The system demonstrated low latency and quick response times under normal load conditions.

Reliability:

The system ensures fault isolation. Failure in one service does not affect the entire system, improving reliability.

Security:

JWT-based authentication ensures secure communication. Unauthorized access is prevented through role-based access control.

Overall, the system performs efficiently and meets the requirements of a modern e-commerce platform.

7. CONCLUSION AND FUTURE SCOPE

The Secure Microservices E-Commerce Platform successfully demonstrates the effectiveness of a microservices-based architecture in developing scalable, secure, and maintainable backend systems. By decomposing the application into independent services such as User, Product, Order, and Payment services, the system achieves loose coupling and improved flexibility. Each service operates independently, allowing efficient handling of user requests and enabling selective scaling based on demand. The use of an API Gateway simplifies client interaction by providing a single entry point, while the Eureka Server enables dynamic service discovery, ensuring seamless communication between services. The implementation of JWT-based authentication and role-based access control enhances system security by protecting sensitive operations and preventing unauthorized access. The integration of an AI-powered chatbot using Large Language Models (LLM) and Retrieval-Augmented Generation (RAG) adds an intelligent layer to the system, improving user interaction and providing real-time assistance. This demonstrates the practical combination of microservices and artificial intelligence in modern application development. Comprehensive testing, including unit testing, API testing, and end-to-end validation, confirms that the system performs reliably and efficiently under different scenarios. Overall, the project meets its objectives of scalability, security, and reliability, making it suitable for real-world e-commerce applications.

Future Scope: The system can be further enhanced by deploying it on cloud platforms such as AWS or Azure to achieve higher scalability and availability. Advanced monitoring and logging tools can be integrated to improve system observability and performance tracking. The AI chatbot can be improved by training it on domain-specific data to provide more accurate and personalized responses. Additionally, features such as recommendation systems and real-time analytics can be added to enhance user experience. The system can also be extended to support mobile applications and real-time notifications using technologies such as WebSockets. Integration with external payment gateways and third-party services can further improve functionality. Overall, the project provides a strong foundation that can be expanded into a full-scale production-ready e-commerce platform.

8. Advantages of the System

The proposed microservices-based e-commerce platform offers several significant advantages when compared to traditional monolithic architectures. One of the most important benefits is scalability. Since each service operates independently, it can be scaled individually based on demand. For instance, if the Product Service experiences higher traffic, it can be scaled without affecting other services such as the User or Payment services. This selective scalability improves resource utilization and system performance.

Another major advantage is flexibility in development and deployment. Each microservice is developed as an independent unit, allowing developers to work on different services simultaneously. This reduces development time and simplifies updates, as changes in one service do not require redeployment of the entire system. This modular approach also makes the system highly adaptable to future requirements.

Fault isolation is another critical benefit of this architecture. In a monolithic system, a failure in one component can affect the entire application. However, in this system, failures are isolated within individual services. For example, if the Payment Service encounters an issue, other services such as Product browsing and User authentication continue to function normally, ensuring system stability.

Security is enhanced through the implementation of JWT-based authentication and role-based access control. Each request is validated at the API Gateway level, ensuring that only authorized users can access system resources. This reduces the risk of unauthorized access and protects sensitive user and transaction data.

The system also offers improved maintainability due to its modular design. Each service can be updated, debugged, or replaced independently without affecting the overall system. This simplifies long-term maintenance and reduces system downtime.

In terms of performance, the system is capable of handling multiple user requests efficiently due to distributed processing. Additionally, the integration of an AI chatbot enhances user experience by providing real-time assistance and improving customer engagement.

Overall, the system demonstrates how microservices architecture can provide a scalable, secure, and efficient solution for modern e-commerce applications.

9. Limitations of the System

Despite the numerous advantages offered by the microservices architecture, the system also has certain limitations that must be considered. One of the primary challenges is the increased complexity associated with managing multiple services. Unlike monolithic systems, where all functionalities are centralized, microservices require coordination between several independent components, making system management more complex.

Another limitation is the need for proper monitoring and management tools. Since the system consists of multiple services running independently, it becomes essential to monitor each service to ensure smooth operation. Without proper monitoring, identifying and resolving issues can become difficult.

The initial setup and development effort for a microservices-based system is also higher compared to traditional architectures. Developers need to configure service discovery, API gateways, and inter-service communication, which adds to the development complexity.

Additionally, the performance of the AI chatbot module depends heavily on the quality and availability of training data. If the chatbot does not have access to relevant data, it may generate inaccurate or less useful responses, affecting user experience.

Network latency can also be a concern, as communication between services occurs over network calls. This can slightly increase response time compared to monolithic systems where all components run within the same application.

Overall, while microservices provide significant benefits, careful planning and management are required to handle these challenges effectively.

10. Real-World Applications

The developed system has several real-world applications, particularly in the domain of modern digital commerce and distributed systems. One of the most prominent applications is in large-scale e-commerce platforms such as Amazon and Flipkart, where microservices architecture is used to handle millions of users and transactions efficiently.

The system can also be used by startups and small businesses that require scalable backend solutions. By adopting microservices architecture, these organizations can build flexible and cost-effective systems that can grow with their user base.

In enterprise environments, the architecture can be applied to develop distributed applications that require high availability and fault tolerance. The independent nature of microservices makes it suitable for large organizations with complex system requirements.

The integration of AI chatbot functionality also makes the system useful for customer support applications. Businesses can use such chatbots to automate responses, reduce human workload, and improve customer satisfaction.

Overall, the system demonstrates how modern technologies such as microservices and artificial intelligence can be combined to build efficient and scalable real-world applications.

References

- [1] Spring Boot Documentation – <https://spring.io/projects/spring-boot>
- [2] Spring Security Documentation – <https://spring.io/projects/spring-security>
- [3] JSON Web Tokens (JWT) Documentation – <https://jwt.io/introduction>
- [4] Netflix Eureka Documentation – <https://github.com/Netflix/eureka>
- [5] JUnit Documentation – <https://junit.org/junit5/>
- [6] Mockito Documentation – <https://site.mockito.org/>
- [7] Postman API Testing – <https://www.postman.com/>
- [8] MySQL Documentation – <https://dev.mysql.com/doc/>
- [9] Retrieval-Augmented Generation (RAG) – Research literature on LLM-based retrieval systems.
- [10] Research papers on microservices architecture and distributed system design.

Appendices

The appendices section provides supplementary information related to the project, including repository details, copyright information, and supporting outputs.

Appendix A – GitHub Repository

Appendix A contains the GitHub repository link where the complete source code of the project is hosted. The repository includes all microservices, configuration files, and documentation required to run the system.

Appendix B – Copyright Filing Details

Appendix B includes the copyright filing details for the project. The system has been submitted under the category of computer software, and a diary number has been generated as proof of submission. The current status of the application is under scrutiny, indicating that it is being reviewed by the concerned authority.

Appendix C – System Output Screenshots

Appendix C consists of system output screenshots that demonstrate the working of the application. These include API responses for user authentication, product listing, order placement, and payment processing. Screenshots of chatbot interactions are also included to show the functionality of the AI module.

These appendices serve as supporting evidence of the system's implementation and validate the results discussed in the report.